Lesson 9 Worksheet: Final Project Planning — ANSWER KEY

Part A: System Design

1. Diagram answers:

- `compute_path()` returns: A list of (row, col) tuples representing each position along the path
- What two things does `drive_path()` do at each step?
  1. Turn the robot to face the needed heading (using turn_to)
  2. Follow the line to the next intersection (using track_until_cross + straight(8) to clear)

---

2. After the Navigator drives one leg, what must you update?

You must update `manhattan.position = navigator.position`. Without this, Manhattan will compute the next path starting from the original position (0, 0) instead of from where the robot actually is now.

---

3. Flow diagram blanks:

- Create Manhattan at (0, 0)
- Create Navigator at (0, 0) heading 0
- Compute path using Manhattan
- Drive path using Navigator
- Update manhattan.position = navigator.position

Part B: Destination Planning (Example Answer)

Note: Student answers will vary. This example uses destinations [(2, 0), (2, 3), (0, 3), (0, 0)].

Order	Destination	Description
Start	(0, 0)	Home base
1	(2, 0)	Bottom-left area
2	(2, 3)	Bottom-right corner area
3	(0, 3)	Top-right corner
4	(0, 0)	Return home

Python code: `destinations = [(2, 0), (2, 3), (0, 3), (0, 0)]`

Step counts:

- Leg 1: (0, 0) to (2, 0): 2 steps
- Leg 2: (2, 0) to (2, 3): 3 steps
- Leg 3: (2, 3) to (0, 3): 2 steps
- Leg 4: (0, 3) to (0, 0): 3 steps
- Total: 10 steps

Part C: Hand-Trace One Leg (Example: Leg 1)

From: (0, 0) to: (2, 0)

Computed path: [(0, 0), (1, 0), (2, 0)]

Headings: 0 = North, 1 = East, 2 = South, 3 = West Right turns = (needed - current) % 4

Step	From	To	Needed Heading	Current Heading	Right Turns	New Heading
1	(0, 0)	(1, 0)	2 (S)	0 (N)	(2-0)%4 = 2	2 (S)
2	(1, 0)	(2, 0)	2 (S)	2 (S)	(2-2)%4 = 0	2 (S)

After this leg:

- Navigator position: (2, 0)
- Navigator heading: 2 (S)

- Manhattan position must be updated to: (2, 0)

What heading will the Navigator have at the START of leg 2? 2 (S) (the heading carries over)

Part D: Testing Checklist

All items should be checked off. For Level 1 sample output:

```
=== XRP Grid Navigation: Final Project ===
Starting at: (0, 0)
Destinations: [(2, 0), (2, 3), (0, 3), (0, 0)]

--- Navigating to (2, 0) ---
Path: [(0, 0), (1, 0), (2, 0)]
Steps: 2
Arrived at: (2, 0)
Heading: S

--- Navigating to (2, 3) ---
Path: [(2, 0), (2, 1), (2, 2), (2, 3)]
Steps: 3
Arrived at: (2, 3)
Heading: E

--- Navigating to (0, 3) ---
Path: [(2, 3), (1, 3), (0, 3)]
Steps: 2
Arrived at: (0, 3)
Heading: N

--- Navigating to (0, 0) ---
Path: [(0, 3), (0, 2), (0, 1), (0, 0)]
Steps: 3
Arrived at: (0, 0)
Heading: W

=== All destinations reached! ===
Final position: (0, 0)
```

Part E: Code Template

The completed main program:

```python
# Create a Board object for wait_for_button()
board = Board.get_default_board()

# Create a Manhattan object starting at (0, 0)
manhattan = Manhattan((0, 0))

# Create a Navigator object starting at (0, 0) heading North (0)
navigator = Navigator((0, 0), 0)

# Define your list of 4+ destinations
destinations = [(2, 0), (2, 3), (0, 3), (0, 0)]

print("=== XRP Grid Navigation: Final Project ===")
print("Starting at:", manhattan.position)
print("Destinations:", destinations)
print()

# Wait for button press before starting
board.wait_for_button()

# Write a for loop that goes through each destination
for dest in destinations:
    print("--- Navigating to", dest, "---")

    # Compute the path using manhattan
    path = manhattan.compute_path(dest)

    print("Path:", path)
    print("Steps:", len(path) - 1)

    # Drive the path using navigator
    navigator.drive_path(path)

    # Update manhattan's position to match navigator's position
    manhattan.position = navigator.position

    print("Arrived at:", navigator.position)
    print("Heading:", HEADING_NAMES[navigator.heading])
    print()

print("=== All destinations reached! ===")
print("Final position:", navigator.position)
```

Blanks in order:

1. `Board.get_default_board()`
2. `Manhattan((0, 0))`
3. `Navigator((0, 0), 0)`

4. `[(2, 0), (2, 3), (0, 3), (0, 0)]` *(answers will vary)*
5. `board.wait_for_button()`
6. `dest` *in* `destinations`
7. `dest`
8. `manhattan.compute_path(dest)`
9. `navigator.drive_path(path)`
10. `manhattan.position = navigator.position`

Reflection

What was the hardest part of putting the whole system together?

Sample answer: The hardest part is usually remembering to update `manhattan.position` *after each leg. Without that line, every path computation starts from (0, 0) regardless of where the robot actually is, so the second leg's path is wrong and the robot drives to the wrong place. The testing strategy of Level 1 (Manhattan only, no robot) helps catch this – you can see in the print output whether each path starts from the correct position. Another common difficulty is debugging physical robot movement: the robot may drift slightly on turns, causing it to miss grid lines. Testing with a single leg first (Level 2) before running the full sequence (Level 3) helps isolate these physical issues from logic bugs.*